Data Structures and Its Types

Overview

A Data Structure is a method of organizing and storing data for efficient use. An Algorithm is a step-by-step procedure used to solve a specific problem. Together, they form the foundation of efficient Computer Science and software design.

Data Structures

Primitive vs. Non-Primitive

Primitive data types are basic built-in types (e.g., integers, booleans, characters). *Non-primitive data types* are more complex structures built from primitives, including both linear and non-linear types.

Linear Data Structures

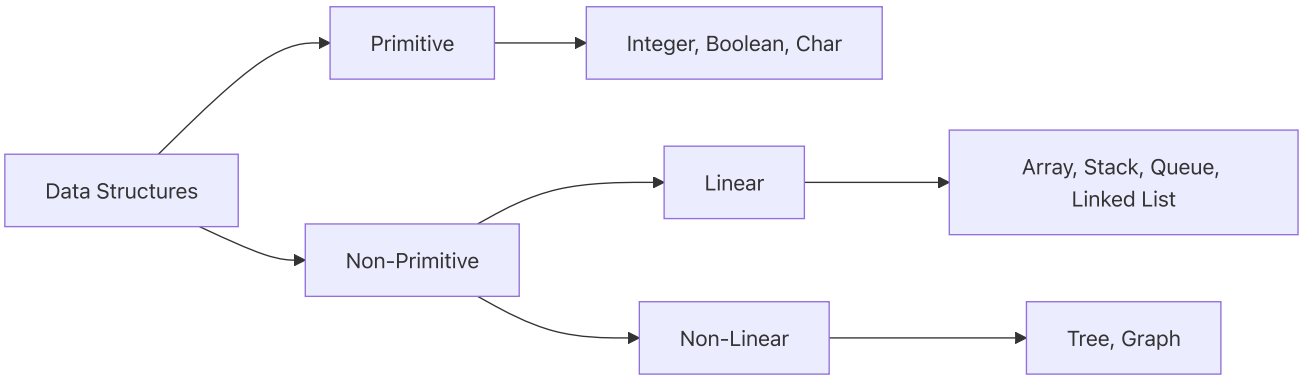
Data elements are arranged in a sequential or linear order.

Structure	Description	Access Pattern
Array	A collection of items stored at contiguous memory locations.	Random Access
Linked List	Elements point to the next element, forming a sequential chain.	Sequential
Stack	A collection where items are added and removed from the top.	LIFO (Last In, First Out)
Queue	A collection where items are added at the back and removed from the front.	FIFO (First In, First Out)

Non-Linear Data Structures

Data elements are not arranged in a sequence; they are interconnected or hierarchical.

- Tree: A hierarchical structure with a root value and subtrees of children.
- Graph: A set of nodes (vertices) connected by edges.



Algorithms

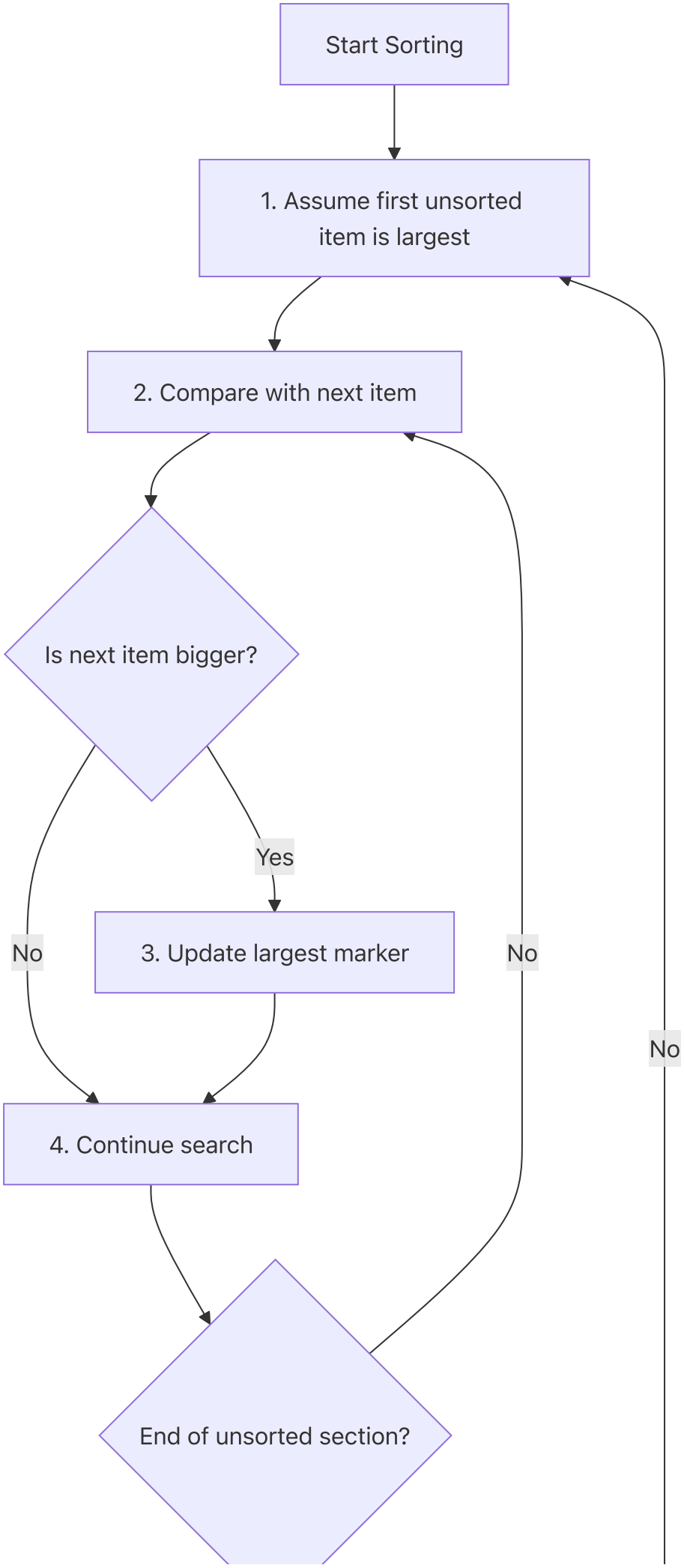
> Definition
>
> An algorithm is a step-by-step procedure to solve a problem and describe it in a real-world language.

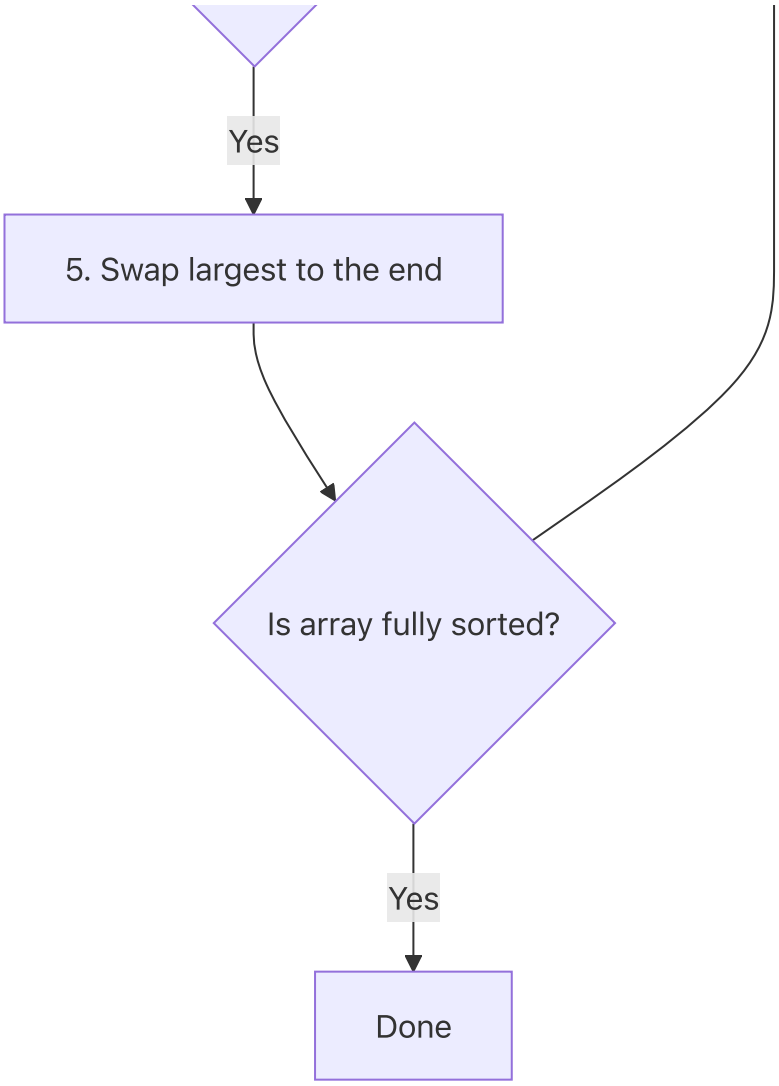
Example: Selection Sort

> Sorting an Array
>
> Selection sort works by repeatedly finding the largest (or smallest) element from the unsorted part and moving it to its correct sorted position.

Step-by-Step Workflow:

1. **Assume the largest:** Take the first item in the unsorted portion of the Array and temporarily assume it is the *largest* value.
2. **Compare with the rest:** Compare this assumed largest value with the next item in the array.
3. **Update the largest:** If you find an item that is bigger than your current assumed largest, update your "largest" marker to point to this new item.
4. **Complete the search:** Continue this comparison process until you have checked every remaining item in the unsorted portion of the array.
5. **Swap into place:** Once you reach the end, swap the true largest item you just found with the item at the very end of your unsorted section.
6. **Repeat for the rest:** Lock that largest item in place (it is now sorted), go back to the remaining unsorted items, and repeat Steps 1–5 until the whole array is sorted.


Start Sorting
1. Assume first unsorted item is largest
2. Compare with next item
Is next item bigger?
Yes
No
3. Update largest marker
No
4. Continue search
End of unsorted section?
No


Yes
5. Swap largest to the end
Is array fully sorted?
Yes
Done

Stacks

Overview

A Stack is a Linear Data Structure that operates strictly on a LIFO (Last In, First Out) or FILO (First In, Last Out) sequence. You can only add or remove elements from one single end, which is called the *top*.

Primary Uses

Stacks are heavily used by computers to manage recursive function calls and for Serialization (processing items one by one in a strict order).

Core Operations

A stack relies on three main commands to control its data.

Operation	Description
Push	Adds a new element to the *top* of the stack.
Pop	Deletes and returns the element currently at the *top* of the stack.
Peek	Returns the value of the *top* element without deleting it.

Mathematical Expression Formatting

Computers use stacks to read and calculate math equations quickly. This method removes the need for parentheses because the order of operations is built directly into the format.

Format Type	Notation Name	Rule	Example
Infix Notation	Standard Notation	Operators sit *between* the numbers.	$A + B$
Prefix Notation	Polish Notation	Operators sit *before* the numbers.	$+AB$
Postfix Notation	Reverse Polish Notation	Operators sit *after* the numbers.	$AB+$

Evaluation Algorithms

1. Postfix Evaluation

To solve a postfix equation, follow this exact path:

1. Scan the expression from **left to right**.
2. If you see a number (Operand), **push** it onto the stack.
3. If you see a math symbol (Operator), **pop** the top two numbers.
4. *Rule: The first popped value is the right side of the equation, the second popped value is the left side.*
5. Calculate the math and **push** the answer back onto the stack.
6. The final answer is the last number left in the stack.

> **Postfix Trace**
>
> **Evaluate:** $AB/C-DE*+$
> (Where $A=12$, $B=6$, $C=3$, $D=4$, $E=5$)
>
> 1. Read $12, 6$: Push 12, Push 6.
> 2. Read $/$: Pop 6, Pop 12. Compute $12/6=2$. Push 2.
> 3. Read 3: Push 3.
> 4. Read $-$: Pop 3, Pop 2. Compute $2-3=-1$. Push -1.
> 5. Read $4, 5$: Push 4, Push 5.
> 6. Read $*$: Pop 5, Pop 4. Compute $4\times5=20$. Push 20.
> 7. Read $+$: Pop 20, Pop -1. Compute $-1+20=19$. Push 19.
>
> **Final Answer:** 19

2. Prefix Evaluation

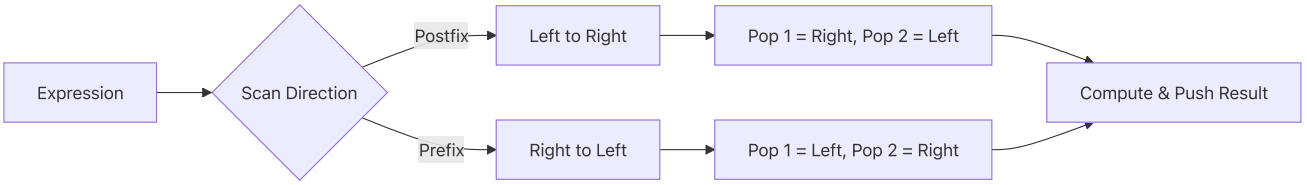
To solve a prefix equation, reverse two critical steps from the postfix method:

1. Scan the expression **backward** (from right to left).
2. When you pop two numbers for a math symbol, *the first popped value is the left side, and the second popped value is the right side*.

> **Prefix Trace**
>
> **Evaluate:** $+*3\ 4\ 5$
>
> 1. Scan Right-to-Left: $5\ 4\ 3\ *\ +$
> 2. Read 5: Push 5 (Stack: [5])
> 3. Read 4: Push 4 (Stack: [5, 4])

4. Read 3: Push 3 (Stack: [5, 4, 3])
5. Read $*$: Pop 3 (left), Pop 4 (right). Compute $3 \times 4 = 12$. Push 12 (Stack: [5, 12])
6. Read $+$: Pop 12 (left), Pop 5 (right). Compute $12 + 5 = 17$. Push 17 (Stack: [17])

Final Answer: 17



Java Implementations

1. Built-in Java Stack

Java has a ready-to-use stack tool inside the `java.util` package. You must use wrapper classes (like `Integer` instead of `int`) for basic data types.

```
import java.util.Stack;

public class Demo {
    public static void main(String[] args) {
        // Must use the Integer wrapper class for primitive ints
        Stack<Integer> myStack = new Stack<>();

        myStack.push(10);
        myStack.push(20);
        myStack.pop();
        myStack.peek();
    }
}
```

2. Custom Array-Based Stack

If you want to build a stack from scratch, you can use a fixed-size Array and an integer Pointer named `top` to keep track of the active boundary.

```
import java.util.*;

public class CustomStack {
    private int top;
    private int maxSize;
    private int[] StackArray;

    public CustomStack(int size) {
```

```
        this.top = -1; // Always initialize top with -1
        this.maxSize = size;
        this.StackArray = new int[maxSize];
    }

    public void push(int el) {
        if(isFull()) {
            System.out.println("Stack Overflow. Cant insert " + el);
            return;
        }
        // Move pointer up, then save the value
        StackArray[++top] = el;
    }

    public int pop() {
        if(isEmpty()) {
            System.out.println("Stack underflow, Nothing to del");
            return -1;
        }
        // Return the value, then move pointer down
        return StackArray[top--];
    }

    public int peek() {
        if(isEmpty()) {
            System.out.println("Stack is Empty");
            return -1;
        }
        return StackArray[top];
    }

    public boolean isFull() {
        // Checks if pointer reached the end of the array
        return top == maxSize - 1;
    }

    public boolean isEmpty() {
        // Checks if stack is completely empty
        return top == -1;
    }
}
```

Stack-Based String Reversal (LIFO Trace)

Core Principle

Why Reversal Happens

A stack only allows access at the top. The *last* character pushed is therefore the *first* character popped. This structural constraint is what forces reversal, with no extra sorting logic required.

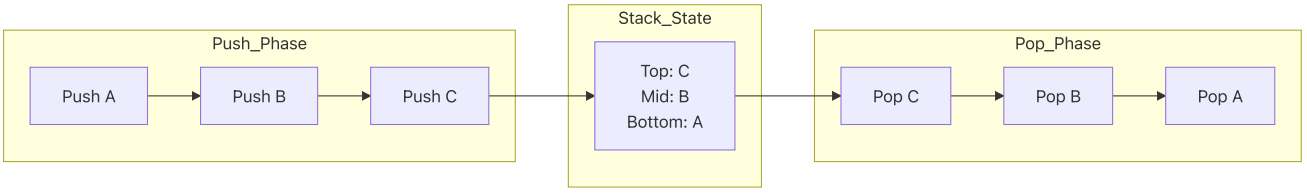
Trace Table

Original String	Push Sequence (Left to Right)	Stack State `[Top -> Bottom]`	Pop Sequence (Reversed Output)
`ABC`	Push `A`, Push `B`, Push `C`	`[C, B, A]`	Pop `C`, Pop `B`, Pop `A` → **`CBA`**
`DMC`	Push `D`, Push `M`, Push `C`	`[C, M, D]`	Pop `C`, Pop `M`, Pop `D` → **`CMD`**
`0709`	Push `0`, Push `7`, Push `0`, Push `9`	`[9, 0, 7, 0]`	Pop `9`, Pop `0`, Pop `7`, Pop `0` → **`9070`**
`2008`	Push `2`, Push `0`, Push `0`, Push `8`	`[8, 0, 0, 2]`	Pop `8`, Pop `0`, Pop `0`, Pop `2` → **`8002`**

Verification

All four rows check out mechanically: the last-pushed element in each case is the first one listed at the top of the stack, and the pop sequence reads that stack top-to-bottom. The logic holds regardless of whether the tokens are letters or digits.

Visualizing One Trace (`ABC` to `CBA`)



The Code

```
public static String reverseUsingStack(String input) {
    stacks stack = new stacks(input.length());

    for (int i = 0; i < input.length(); i++) {
        stack.push(input.charAt(i));
    }

    String reversedString = "";
    while (!stack.isEmpty()) {
        reversedString += (char) stack.pop();
    }

    return reversedString;
}
```

Related Concepts

- Push: Adds an element to the top of the stack.
- Pop: Removes and returns the top element.
- Peek: Reads the top element without removing it.
- Postfix Evaluation Algorithm: Another example of stacks processing tokens left to right.
- Prefix Evaluation Algorithm: Uses the same LIFO mechanics but scans right to left.

Infix to Prefix and Postfix

Overview

Computers and humans evaluate mathematical expressions differently. Humans rely on visual groupings and strict Operator Precedence rules. Computers process data sequentially using a Stack data structure. This note explains how to translate human-readable math into machine-readable formats.

The Three Mathematical Notations

We categorize mathematical expressions into three primary formats based on where the operator is placed relative to its operands.

Notation Type	Structure	User Base	Evaluation Logic	Parentheses Required?
Infix	Operator is *between* operands (e.g., A + B)	Humans	Requires strict precedence rules	Yes, to resolve ambiguities
Postfix (Reverse Polish)	Operator *follows* operands (e.g., A B +)	Machines	Strictly left-to-right using LIFO logic	No
Prefix (Polish Notation)	Operator *precedes* operands (e.g., + A B)	Machines	Strictly right-to-left evaluation	No

---

The Operator Precedence Hierarchy

When converting expressions using a Stack, the stack acts as a holding cell for Operators. Precedence dictates which operator gets to stay in the stack and which gets forced out to the output.

Precedence Levels

1. Highest: `^` (Exponentiation)
2. Medium: `*` (Multiplication), `/` (Division)

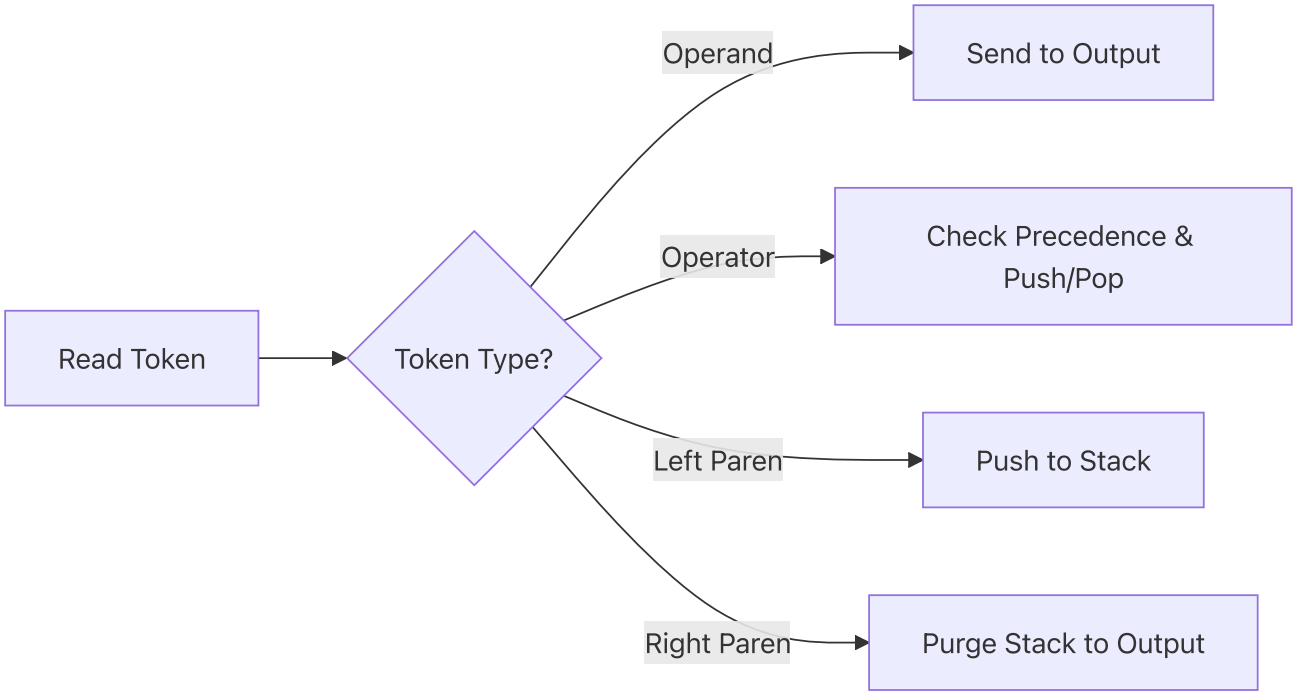
3. **Lowest:** `+` (Addition), `-` (Subtraction)

Infix to Postfix Conversion

The Shunting-yard Algorithm reads the Infix string token by token from left to right.

Core Algorithm Rules

1. Operands (Letters/Numbers): Always bypass the stack and go directly to the output.
2. Operators (`+`, `-`, `*`, `/`, `^`): Go to the stack, but must obey the **Precedence Check**.
   - *Precedence Check:* If the incoming operator has lower or equal precedence than the top stack operator, the stack operator is popped to the output. Repeat until the stack top is strictly lower precedence or empty. Then push the new operator.
3. **Parentheses** (The Overrides):
   - *Opening* `(` : Pushed directly to the stack, ignoring precedence. Acts as a concrete floor.
   - *Closing* `)` : Never goes into the stack. Triggers a stack purge. Pop every operator to the output until the matching `(` is encountered. Discard both parentheses.
4. **End of String**: Once fully read, pop all remaining operators from the stack to the output.



Simple Trace Examples

Trace A: `A + B * C`

Higher precedence `stacks safely on lower precedence` `+` `*`

1. Read `A` → Output: `A`
2. Read `+` → Stack: `[+]`
3. Read `B` → Output: `A B`
4. Read `*` → `*` > `+`. Stack: `[+, *]`
5. Read `C` → Output: `A B C`
6. End → Pop all. Output: `A B C * +`

Trace B: `(A + B) * C`

Parentheses force addition to evaluate first

1. Read `(` → Stack: `[ ( ]`
2. Read `A` → Output: `A`
3. Read `+` → Stack: `[ (, + ]`
4. Read `B` → Output: `A B`
5. Read `)` → Purge stack until `(`. Pop `+`. Stack: `[ ]`. Output: `A B +`
6. Read `*` → Stack: `[ * ]`
7. Read `C` → Output: `A B + C`
8. End → Pop all. Output: `A B + C *`

Complex Trace Example

Expression: `A + B * C + (D * E + F) * G`

This tests the precedence hierarchy, left-to-right associativity (equal precedence popping), and parenthesis overrides.

Read Token	Stack State	Output String
A	`[ ]`	`A`
+	`[ + ]`	`A`
B	`[ + ]`	`A B`
*	`[ +, * ]`	`A B`
C	`[ +, * ]`	`A B C`
+	`[ + ]`	`A B C * +`
(	`[ +, ( ]`	`A B C * +`
D	`[ +, ( ]`	`A B C * + D`
*	`[ +, (, * ]`	`A B C * + D`
E	`[ +, (, * ]`	`A B C * + D E`

Read Token	Stack State	Output String
+	`[ +, (, + ]`	`A B C * + D E *`
F	`[ +, (, + ]`	`A B C * + D E * F`
)	`[ + ]`	`A B C * + D E * F +`
*	`[ +, * ]`	`A B C * + D E * F +`
G	`[ +, * ]`	`A B C * + D E * F + G`
End	`[ ]`	`A B C * + D E * F + G * +`

Final Postfix Result: `A B C * + D E * F + G * +`

Infix to Prefix Conversion

To convert Infix to Prefix using stack logic, you run a modified three-step process.

1. Reverse the original infix expression. Swap opening and closing parentheses (`(` becomes `)` and vice versa).
2. Convert the reversed string using the standard Infix to Postfix algorithm, but apply the Equal Precedence Tweak.
3. Reverse the final output string.

> ### The Equal Precedence Tweak
>
> Because you reverse the string in Step 1, you are reading the math backward. This changes associativity logic.
>
> - Normal Postfix Rule: Pop if stack precedence is $\geq$ incoming precedence.
> - Reversed Prefix Rule: Pop if stack precedence is $>$ incoming precedence. *If they are equal, push it onto the stack.*

Simple Trace Example A

Original Expression: `A + B * C`

Step 1: Reverse the String and Swap Parentheses
Reversed & Swapped: `C * B + A`

Step 2: Apply the Modified Postfix Algorithm

Read Token	Stack State	Output String
C	`[ ]`	`C`

Read Token	Stack State	Output String
*	`[ * ]`	`C`
B	`[ * ]`	`C B`
+	`[ + ]`	`C B *`
A	`[ + ]`	`C B * A`
End	`[ ]`	`C B * A +`

Step 3: Reverse the Final Output String
Step 2 Output: `C B * A +`
Final Prefix Result: `+ A * B C`

Simple Trace Example B

Original Expression: `(A + B) * C`

Step 1: Reverse the String and Swap Parentheses
Reversed & Swapped: `C * (B + A)`

Step 2: Apply the Modified Postfix Algorithm

Read Token	Stack State	Output String
C	`[ ]`	`C`
*	`[ * ]`	`C`
(	`[ *, ( ]`	`C`
B	`[ *, ( ]`	`C B`
+	`[ *, (, + ]`	`C B`
A	`[ *, (, + ]`	`C B A`
)	`[ * ]`	`C B A +`
End	`[ ]`	`C B A + *`

Step 3: Reverse the Final Output String
Step 2 Output: `C B A + *`
Final Prefix Result: `* + A B C`

Complex Trace Example

Original Expression: `A + B * C + (D * E + F) * G`

Step 1: Reverse the String and Swap Parentheses
Reversed & Swapped: `G * (F + E * D) + C * B + A`

Step 2: Apply the Modified Postfix Algorithm

Read Token	Stack State	Output String
G	`[ ]`	`G`
*	`[ * ]`	`G`
(	`[ *, ( ]`	`G`
F	`[ *, ( ]`	`G F`
+	`[ *, (, + ]`	`G F`
E	`[ *, (, + ]`	`G F E`
*	`[ *, (, +, * ]`	`G F E`
D	`[ *, (, +, * ]`	`G F E D`
)	`[ * ]`	`G F E D * +`
+	`[ + ]`	`G F E D * + *`
C	`[ + ]`	`G F E D * + * C`
*	`[ +, * ]`	`G F E D * + * C`
B	`[ +, * ]`	`G F E D * + * C B`
+	`[ +, + ]`	`G F E D * + * C B *`
A	`[ +, + ]`	`G F E D * + * C B * A`
End	`[ ]`	`G F E D * + * C B * A + +`

Step 3: Reverse the Final Output String

Step 2 Output: `G F E D * + * C B * A + +`

Final Prefix Result: `+ + A * B C * + * D E F G`

Queues

Queues: Linear and Circular

Overview

A **Queue** is a Linear Data Structure that strictly follows a **FIFO** (First In, First Out) sequence. The first item placed into the queue is always the first item removed. Unlike a Stack, which operates from a single end, a queue uses two distinct pointers to manage its boundaries.

Core Operations and Pointers

A queue uses two pointers to keep track of its data inside an Array.

Pointer	Purpose	Action
Front (F)	Tracks the index from which elements are removed.	**Dequeue**
Rear (R)	Tracks the index at which new elements are inserted.	**Enqueue**



---

1. Linear Custom Queue

In a standard linear queue, you insert items at the back and remove them from the front.

Initial State

When the queue is built, the Array is allocated based on a maximum size. Both the Front (`F`) and Rear (`R`) pointers start at `-1` . This mathematically represents that the queue is completely empty.

Java Implementation

```java
public class CustomQueue {
    private int[] array;
    private int F, R;
    private int size;
```

```
    public CustomQueue(int Qsize) {
        this.size = Qsize;
        this.F = -1;
        this.R = -1;
        this.array = new int[size];
    }

    public void enqueue(int item) {
        if (isFull()) {
            System.out.println("Overflow: Queue is full.");
            return;
        } else if (F == -1 && R == -1) {
            F = R = 0;
            array[R] = item;
        } else {
            R++;
            array[R] = item;
        }
    }

    public int dequeue() {
        if (isEmpty()) {
            System.out.println("Underflow: Queue is empty.");
            return -1;
        } else if (F == R) {
            int data = array[F];
            F = R = -1;
            return data;
        } else {
            int data = array[F];
            F++;
            return data;
        }
    }

    public boolean isFull() {
        return R == size - 1;
    }

    public boolean isEmpty() {
        return F == -1;
    }
}
```

2. Circular Queue

A linear queue has a major physical flaw: wasted space. As items are removed, the Front pointer moves forward, leaving empty, unusable slots at the beginning of the array. Once the Rear pointer hits the end of the array, the system treats the queue as full, even if space is available at the front.

A Circular Queue solves this memory issue.

> The Solution
>
> A circular queue uses Modulo Arithmetic (`% size`). This mathematical trick allows the pointers to wrap around back to index `0` when they reach the end of the array. It treats the straight array like an endless circle.

> How the Full Check Works
>
> This implementation has no separate `isFull()` method. Instead, `enqueue` tentatively advances `R` with `(R + 1) % size`, then checks if `R` has collided with `F`. If so, the queue was already full *before* this insertion, so `R` is stepped back to undo the tentative move and the item is rejected. This is logically equivalent to checking `(R + 1) % size == F` before writing, just written as write-then-check-then-undo instead of check-then-write.

Java Implementation

```java
public class CircularQueue {
    private int[] Array;
    private int F, R, size;

    public CircularQueue(int Qsize) {
        this.size = Qsize;
        this.F = -1;
        this.R = -1;
        this.Array = new int[size];
    }

    public void enqueue(int item) {
        if (isEmpty()) {
            F = R = 0;
            Array[R] = item;
        } else {
            // Increment the index in a circle
            R = (R + 1) % size;

            if (R == F) {
                System.out.println("Queue is full");
```

```
                // Step back if full to avoid overwriting the front
                R = (R - 1 + size) % size;
            } else {
                Array[R] = item;
            }
        }
    }

    public int dequeue() {
        int item = -1;

        if (!isEmpty()) {
            item = Array[F];
            if (F == R) {
                // Reset pointers if it was the very last item
                F = R = -1;
            } else {
                // Increment the front pointer in a circle
                F = (F + 1) % size;
            }
        } else {
            System.out.println("Empty Queue");
        }

        return item;
    }

    public boolean isEmpty() {
        return F == -1;
    }
}
```

Linear vs Circular Comparison

Aspect	Linear Queue	Circular Queue
Space Reuse	Wastes space after dequeues; freed front slots are never reused.	Wraps pointers with `% size`, reusing freed slots.
Full Condition	`R == size - 1`, even if front slots are free.	`R` would collide with `F` after wrapping.
Empty Condition	`F == -1`	`F == -1`

Aspect	Linear Queue	Circular Queue
Reset on Last Item	`F = R = -1`	`F = R = -1`

Linked Lists

Linked List Fundamentals and Core Operations

Overview

This note covers the structure of a Linked List, the anatomy of a Node, and six core operations: Append, position-based Insertion, Traversal, Deletion, and Search. The earlier off-by-one defect in the position-based insert has been corrected in this version, and the fix is verified below with a trace rather than taken on faith.

Core Structure

Defining the Pieces

A Linked List is a linear data structure where elements are not stored in contiguous memory. Each element is a Node, dynamically allocated and connected via pointers rather than array indices.

- Data field: holds the actual value
- `next` pointer: holds the memory address of the subsequent Node.
- Head Pointer: a single `head` reference tracks the start of the list.
- Termination: the final Node points to `Null`, marking the end of the chain.

Memory Visualization:

```
[301] -> [A|201] -> [B|2018] -> [C|405] -> [D|Null]
```

The `NODE` Class

```
class CustomLinkedList {
    NODE head;

    static class NODE {
        int data;
        NODE next;

        NODE(int val) {
            this.data = val;
```

```
            this.next = null;
        }
    }
}
```

Inserting at the End (Append)

```
public void insert(int data) {
    NODE newNode = new NODE(data);

    if (head == null) {
        head = newNode;
        return;
    }

    NODE current = head;

    while (current.next != null) {
        current = current.next;
    }

    current.next = newNode;
}
```

Inserting at a Specific Position

```
public void insert(int data, int position) {
    NODE newnode = new NODE(data);

    if (position == 0) {
        newnode.next = head;
        head = newnode;
        return;
    }

    NODE prev = head;

    for (int i = 0; i < position - 1; i++) {
        if (prev != null) {
            prev = prev.next;
        }
    }
```

```
    if (prev != null) {
        newnode.next = prev.next;
        prev.next = newnode;
    } else {
        System.out.println("Out of bounds");
    }
}
```

Traversal (Display)

```
public void display() {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }

    NODE current = head;

    while (current != null) {
        System.out.print(current.data + " -> ");
        current = current.next;
    }
    System.out.println("null");
}
```

🖊 Why `current` Instead of `head`

Traversing with `head` directly would permanently lose the start of the list the moment `head = head.next` runs. Using a disposable `current` pointer is the correct and necessary pattern here.

Deletion (By Value)

```
public void delete(int key) {
    NODE current = head;
    NODE prev = null;

    if (current != null && current.data == key) {
        head = current.next;
        return;
    }
```

```
    while (current != null && current.data != key) {
        prev = current;
        current = current.next;
    }

    if (current == null) {
        System.out.println("Value " + key + " not found in the list.");
        return;
    }

    prev.next = current.next;
}
```

Searching

```
public boolean search(int key) {
    NODE current = head;

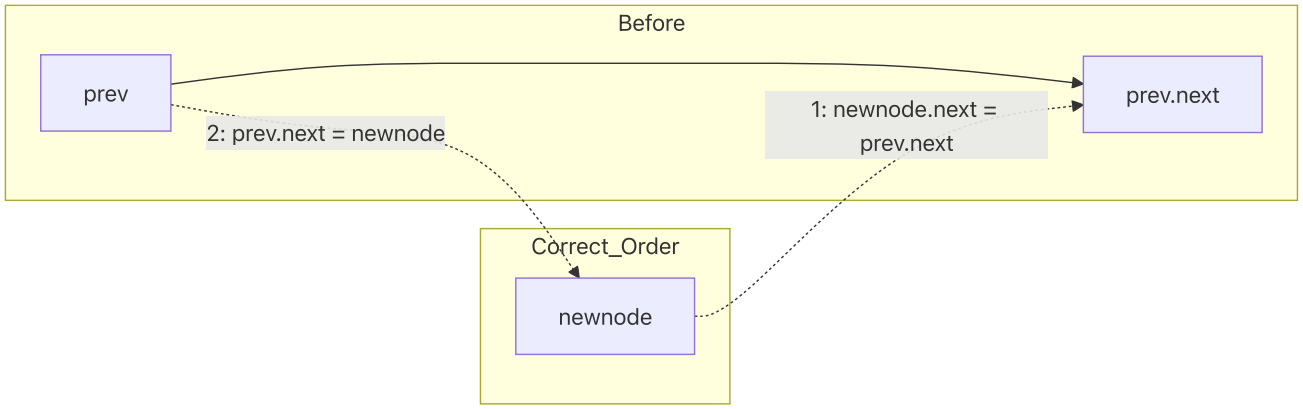
    while (current != null) {
        if (current.data == key) {
            return true;
        }
        current = current.next;
    }

    return false;
}
```

Operation Summary

Operation	Time Complexity	Requires Traversal
Append	*O(n)*	Yes, to find the tail
Insert at Head	*O(1)*	No
Insert at Position	*O(n)*	Yes, up to `position - 1`
Display	*O(n)*	Yes, full list
Deletion	*O(n)*	Yes, worst case
Search	*O(n)*	Yes, worst case

Pointer Flow for Position-Based Insert



Lab Task 1

Linked List Implementations

> **Overview**
>
> This note covers three Java implementations built on the Linked List structure: a Stack, a Queue, and a Doubly Linked List. Each uses a custom `Node` class and manages memory through references instead of a fixed-size Array.

Core Building Block: The Node

Every program starts with a `Node` that stores a value and one or more references to neighbouring nodes.

Structure	Node Fields	Direction
Stack	`data`, `next`	One way, top to bottom
Queue	`data`, `next`	One way, front to rear
Doubly Linked List	`data`, `prev`, `next`	Two way

> **⚠ File Separation**
>
> All three programs declare their own `Node` class. In Java this only compiles if each program lives in its own file or package. Pasted into a single file, the three `Node` declarations would collide.

Question 1: Stack Using Linked List

A Stack follows LIFO order. The `top` reference always points to the most recently pushed node, so every operation happens at one end.



Operations

Method	Behaviour	Time
`push(data)`	Creates a node, links it to the old top, moves `top` to it.	$O(1)$
`pop()`	Returns `top.data`, moves `top` to `top.next`.	$O(1)$
`peek()`	Returns `top.data` without removing it.	$O(1)$
`isEmpty()`	Returns `true` when `top == null`.	$O(1)$
`display()`	Walks from `top` to the end, printing each value.	$O(n)$

Java Implementation

```java
import java.util.Scanner;

class Node {
    int data;
    Node next;
    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}

class Stack {
    Node top;
    public void push(int data) {
        Node newNode = new Node(data);
        newNode.next = top;
        top = newNode;
    }
    public int pop() {
        if (isEmpty()) {
            System.out.println("Stack Empty");
            return -1;
        }
        int popped = top.data;
        top = top.next;
        return popped;
    }
    public int peek() {
        if (isEmpty()) {
            System.out.println("Stack Empty");
            return -1;
        }
        return top.data;
    }
    public boolean isEmpty() {
        return top == null;
```

```
    }
    public void display() {
        if (isEmpty()) {
            System.out.println("Stack Empty");
            return;
        }
        Node current = top;
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.next;
        }
        System.out.println();
    }
}

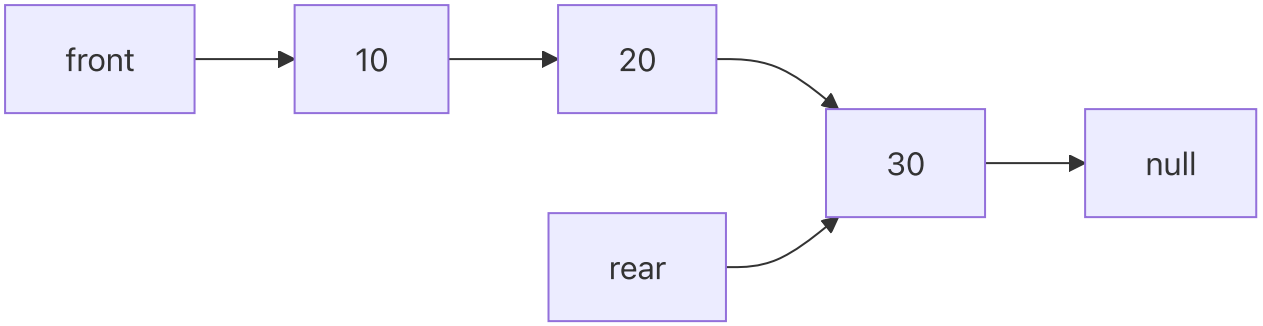
public class StackMenu {
    public static void main(String[] args) {
        Stack stack = new Stack();
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.println("\nSTACK MENU\n1. Push\n2. Pop\n3. Peek\n4.
Check Empty\n5. Display\n6. Exit");
            int choice = sc.nextInt();
            switch (choice) {
                case 1:
                    stack.push(sc.nextInt());
                    break;
                case 2:
                    System.out.println(stack.pop());
                    break;
                case 3:
                    System.out.println(stack.peek());
                    break;
                case 4:
                    System.out.println(stack.isEmpty());
                    break;
                case 5:
                    stack.display();
                    break;
                case 6:
                    System.exit(0);
            }
        }
    }
}
```

🖊 Why No `isFull()`

The array-based version needs an `isFull()` check because its size is fixed. A linked stack grows one node at a time, so it never overflows until the JVM runs out of memory.

Question 2: Queue Using Linked List

A Queue follows FIFO order. It tracks two references: `front` for removals and `rear` for insertions, so both ends are $O(1)$ with no shifting.



Operations

Method	Behaviour	Time
`enqueue(data)`	Appends at `rear`. If the queue is empty, sets both `front` and `rear`.	$O(1)$
`dequeue()`	Returns `front.data`, advances `front`.	$O(1)$
`peek()`	Returns `front.data` without removing it.	$O(1)$
`isEmpty()`	Returns `true` when `front == null`.	$O(1)$
`display()`	Walks from `front` to the end.	$O(n)$

> **The Edge Case in `dequeue()`**
>
> After advancing `front`, the code checks `if (front == null) rear = null;`. Without this line, removing the last item would leave `rear` pointing at a deleted node, and the next `enqueue` would attach to it incorrectly.

Java Implementation

```java
import java.util.Scanner;

class Node {
```

```
    int data;
    Node next;
    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}

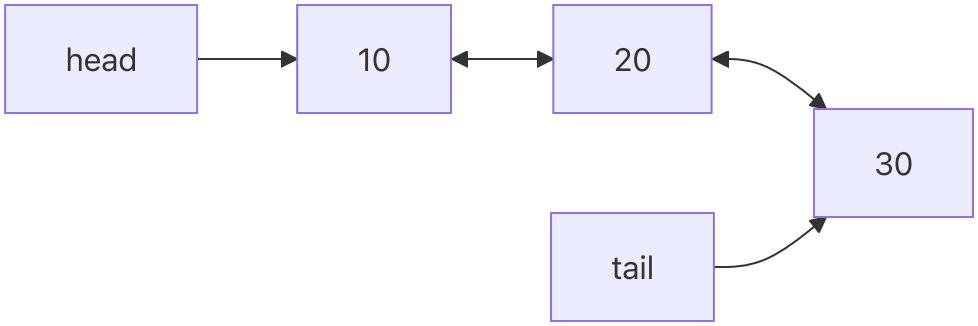
class Queue {
    Node front, rear;
    public void enqueue(int data) {
        Node newNode = new Node(data);
        if (rear == null) {
            front = rear = newNode;
            return;
        }
        rear.next = newNode;
        rear = newNode;
    }
    public int dequeue() {
        if (isEmpty()) {
            System.out.println("Queue Empty");
            return -1;
        }
        int dequeued = front.data;
        front = front.next;
        if (front == null) {
            rear = null;
        }
        return dequeued;
    }
    public int peek() {
        if (isEmpty()) {
            System.out.println("Queue Empty");
            return -1;
        }
        return front.data;
    }
    public boolean isEmpty() {
        return front == null;
    }
    public void display() {
        if (isEmpty()) {
            System.out.println("Queue Empty");
            return;
        }
        Node current = front;
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.next;
```

```
        }
        System.out.println();
    }
}

public class QueueMenu {
    public static void main(String[] args) {
        Queue queue = new Queue();
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.println("\nQUEUE MENU\n1. Enqueue\n2. Dequeue\n3.
Peek\n4. Check Empty\n5. Display\n6. Exit");
            int choice = sc.nextInt();
            switch (choice) {
                case 1:
                    queue.enqueue(sc.nextInt());
                    break;
                case 2:
                    System.out.println(queue.dequeue());
                    break;
                case 3:
                    System.out.println(queue.peek());
                    break;
                case 4:
                    System.out.println(queue.isEmpty());
                    break;
                case 5:
                    queue.display();
                    break;
                case 6:
                    System.exit(0);
            }
        }
    }
}
```

Question 3: Doubly Linked List

A Doubly Linked List gives each node both a `prev` and a `next` reference. The list keeps `head` and `tail` pointers, so it can be traversed in either direction and can remove from the end without scanning.



Operations

Method	Behaviour	Time
`insertAtBeginning(data)`	Links a new node before `head`.	$O(1)$
`insertAtEnd(data)`	Links a new node after `tail`.	$O(1)$
`insertAtPosition(data, position)`	Walks to the node before the target position, then splices in. Falls back to beginning or end when out of range.	$O(n)$
`deleteFromBeginning()`	Moves `head` forward and clears its `prev`.	$O(1)$
`deleteFromEnd()`	Moves `tail` backward and clears its `next`.	$O(1)$
`deleteFromPosition(position)`	Walks to the target node and relinks its neighbours.	$O(n)$
`displayForward()`	Prints from `head` using `next`.	$O(n)$
`displayBackward()`	Prints from `tail` using `prev`.	$O(n)$

The Four-Pointer Splice

Inserting in the middle requires updating four references in the correct order:

1. `newNode.next = current.next`
2. `newNode.prev = current`
3. `current.next.prev = newNode`
4. `current.next = newNode`

Doing step 4 before step 3 would lose the only reference to the following node.

Java Implementation

```
class Node {
    int data;
    Node prev;
```

```
    Node next;
    public Node(int data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}

public class DoublyLinkedList {
    Node head;
    Node tail;

    public DoublyLinkedList() {
        this.head = null;
        this.tail = null;
    }

    public void insertAtBeginning(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }

    public void insertAtEnd(int data) {
        Node newNode = new Node(data);
        if (tail == null) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    public void insertAtPosition(int data, int position) {
        if (position <= 1) {
            insertAtBeginning(data);
            return;
        }
        Node newNode = new Node(data);
        Node current = head;
        for (int i = 1; i < position - 1 && current != null; i++) {
            current = current.next;
        }
        if (current == null || current.next == null) {
```

```
            insertAtEnd(data);
        } else {
            newNode.next = current.next;
            newNode.prev = current;
            current.next.prev = newNode;
            current.next = newNode;
        }
    }

    public void deleteFromBeginning() {
        if (head == null) return;
        if (head == tail) {
            head = tail = null;
        } else {
            head = head.next;
            head.prev = null;
        }
    }

    public void deleteFromEnd() {
        if (tail == null) return;
        if (tail == head) {
            head = tail = null;
        } else {
            tail = tail.prev;
            tail.next = null;
        }
    }

    public void deleteFromPosition(int position) {
        if (head == null) return;
        if (position < 1) return;
        if (position == 1) {
            deleteFromBeginning();
            return;
        }
        Node current = head;
        for (int i = 1; i < position && current != null; i++) {
            current = current.next;
        }
        if (current == null) return;
        if (current == tail) {
            deleteFromEnd();
        } else {
            current.prev.next = current.next;
            current.next.prev = current.prev;
        }
    }

    public void displayForward() {
```

```
        Node current = head;
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.next;
        }
        System.out.println();
    }

    public void displayBackward() {
        Node current = tail;
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.prev;
        }
        System.out.println();
    }
}
```

Design Notes and Limitations

Issue	Where	Why It Matters
`-1` as error value	Stack and Queue	`-1` is a legitimate data value, so callers cannot distinguish an empty structure from a stored `-1`. An exception or wrapper type avoids this.
No input validation	Both menus	`sc.nextInt()` throws `InputMismatchException` on non-numeric input.
No default case	Both menus	Entering a number outside 1 to 6 silently redraws the menu.
Scanner never closed	Both menus	`System.exit(0)` ends the program, so it is harmless here, but it is poor practice in larger programs.
No `main` method	Doubly Linked List	The class cannot be run on its own and needs a separate driver to test it.

Comparison: Stack vs Queue vs Doubly Linked List

Aspect	Stack	Queue	Doubly Linked List
Order	LIFO	FIFO	Arbitrary
Pointers Kept	`top`	`front`, `rear`	`head`, `tail`

Aspect	Stack	Queue	Doubly Linked List
Insert Point	Top only	Rear only	Beginning, end, or any position
Remove Point	Top only	Front only	Beginning, end, or any position
Traversal	One direction	One direction	Both directions
Memory per Node	1 reference	1 reference	2 references

Time Complexity

Time Complexity Fundamentals

Overview

This note covers Time Complexity and Asymptotic Notation, the mathematical language used to describe how an algorithm's running time scales with input size. It walks through five growth patterns: Constant Time, Linear Time, Sequential Loops, Quadratic Time, and Logarithmic Time.

Why We Use Asymptotic Notation

The Core Idea

Time Complexity measures how an algorithm's runtime scales, not its runtime in literal seconds. Measuring seconds is unreliable because faster hardware finishes sooner, so the same algorithm would appear to have different "speeds" on different machines. Asymptotic Notation solves this by describing growth trends as input size approaches infinity, independent of hardware.

The Three Notations

Notation	Bound Type	Meaning
Big O Notation (O)	Upper bound	*Worst-case* scenario, the longest an algorithm could take
Big Omega Notation (Ω)	Lower bound	*Best-case* scenario, the fastest an algorithm could run
Big Theta Notation (Θ)	Tight bound	Big Theta gives you the exact rate. It means the program's speed will consistently scale at this precise pace. no faster, no slower. In simple terms, it tells you exactly how a computer program's running time grows as the workload increases

---

Calculating Big O: Rules and Examples

> **The Golden Rule**
>
> Add up the time for each line, then drop constants and lower-order terms, since they stop mattering as input size grows toward infinity.

1. Constant Time: $O(1)$

Simple steps like variable assignment, printing, or returning a value take the same time regardless of input size.

```
class Main {
    public static void main(String[] args) {
        int n = 10;              // O(1) time
        System.out.println(n);  // O(1) time
    }
}
```

Calculation: $O(1) + O(1) + O(1) = O(3)$. Since 3 is a fixed constant, it is dropped. The final complexity is $O(1)$.

---

2. Linear Time: $O(n)$

A loop that runs n times has a runtime that grows exactly as fast as n.

```
class Main {
    public static void main(String[] args) {
        int n = 10;

        for (int i = 1; i <= n; i++) {        // Loop runs n times
            System.out.println(n);            // O(1) operation inside an
O(n) loop
        }
    }
}
```

Calculation: $O(1) + O(n)$. Since $O(n)$ dominates as n grows, the constant $O(1)$ is dropped. The final complexity is $O(n)$.

---

3. Sequential Loops: $O(n + m)$

Two separate loops over different variables, run one after another, have their complexities added.

```
class Main {
    public static void main(String[] args) {
        int n = 10;
        int m = 5;

        // First loop relies on n
        for (int i = 1; i <= n; i++) {        // O(n)
            System.out.println(n);
        }

        // Second loop relies on m
        for (int j = 1; j <= m; j++) {        // O(m)
            System.out.println(m);
        }
    }
}
```

Calculation: $O(n) + O(m) = O(n + m)$.

---

4. Quadratic Time (Nested Loops): $O(n^2)$

A loop nested inside another loop multiplies their complexities, rather than adding them.

```
class Main {
    public static void main(String[] args) {
        int n = 10;

        // Outer loop runs n times
        for (int i = 1; i <= n; i++) {
            System.out.println(n);

            // Inner loop runs n times FOR EACH outer iteration
            for (int j = 1; j <= n; j++) {
                // ...
            }
        }
    }
}
```

Calculation: $O(n) \times O(n) = O(n^2)$.

---

5. Logarithmic Time: $O(\log n)$

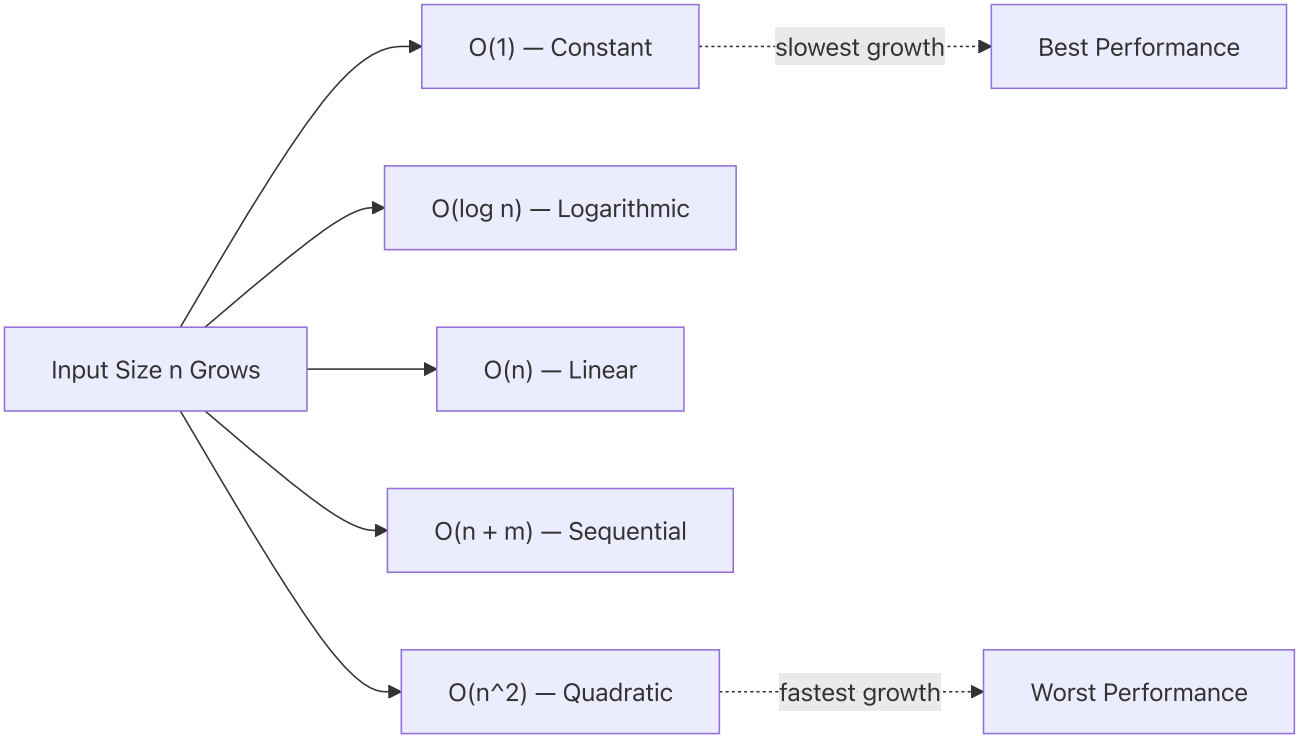
This occurs when the loop counter is multiplied or divided at each step, rather than incremented by a fixed amount. This sharply reduces the number of iterations needed, even for very large n. It appears frequently in Divide and Conquer algorithms such as Binary Search.

```
class Main {
    public static void main(String[] args) {
        int n = 1000;

        for (int i = 1; i < n; i = i * 2) {   // i doubles each step
            System.out.println(i);
        }
    }
}
```

Calculation: The loop counter doubles every iteration, so the number of iterations is roughly $\log_2 n$. The final complexity is $O(\log n)$.

Growth Rate Comparison



Searching Algorithms

> **Overview**
>
> This note covers two families of algorithms. Searching Algorithms find a value or index inside a data structure, using either Linear Search or Binary Search. Complexity is described using Big O Notation.

Searching Algorithms

Searching is the process of finding a specific number, value, or index in a given data structure. There are two primary types:

- Linear Search (also called *Sequential Search*)
- Binary Search

Linear Search

Linear search checks every element in the array one by one, from start to finish, until it finds the target.

- **Data requirement:** works on both *sorted and unsorted* arrays.
- **How it works:** it visits each index in order.

Case	Time Complexity	Explanation
Worst case	$O(n)$	The target is at the very end or not in the array at all
Best case	$O(1)$	The target is at the first index, `0`
Space	$O(1)$	No extra memory is needed

```java
public int linearSearch(int[] arr, int target) {
    // Traverse each index one by one
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            System.out.println("Number found");
            return i; // Return the index where it was found
        }
    }
```

```
    return -1; // Target does not exist in the array
}
```

Binary Search

Binary search repeatedly halves the search space by comparing the target with the middle element of the array.

> **Sorted Data Is Mandatory**
>
> The array must be sorted first. On unsorted data, binary search gives unreliable results, because discarding half the array is only valid when the order is known.

How it works:

1. Set a `left` pointer at the start (`0`) and a `right` pointer at the end (`array.length - 1`).
2. Find the `mid` index halfway between `left` and `right`.
3. If `array[mid]` equals the target, return `mid`.
4. If `array[mid] < target`, the target must be in the right half. Move `left` to `mid + 1`.
5. If `array[mid] > target`, the target must be in the left half. Move `right` to `mid - 1`.

Case	Time Complexity	Explanation
Worst case	$O(\log n)$	The search area is cut in half at every step
Best case	$O(1)$	The target is the first middle element checked
Space (iterative)	$O(1)$	Only a few pointer variables are used

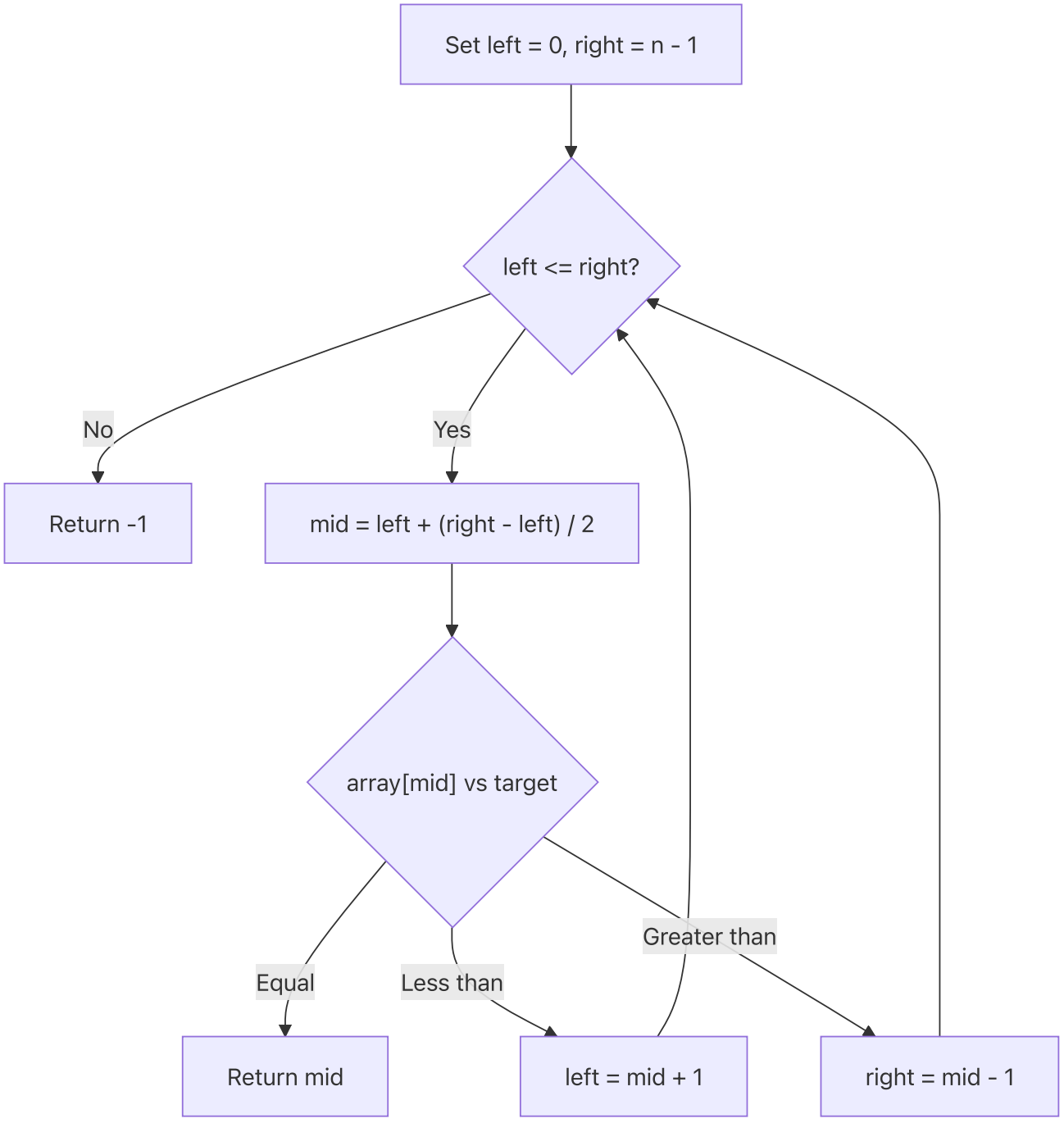
```
public int binarySearch(int[] array, int target) {
    int left = 0;
    int right = array.length - 1;

    while (left <= right) {
        // Overflow-safe middle index
        int mid = left + (right - left) / 2;

        // Case 1: Target is at the middle
        if (array[mid] == target) {
            return mid;
        }
        // Case 2: Target is larger than mid, so search the right half
        else if (array[mid] < target) {
```

```
            left = mid + 1;
        }
        // Case 3: Target is smaller than mid, so search the left half
        else {
            right = mid - 1;
        }
    }

    return -1; // Target not found after loop finishes
}
```



Binary Search Dry Run

Visual Concept

Splitting `[10, 20, 30, 40, 50, 60]` in half gives `[10, 20, 30]` and `[40, 50, 60]`. If the target is `40`, it is in the right half, so the left half is discarded in one comparison.

Setup:

- **Array (9 elements, indices `0` to `8`):** `[10, 20, 30, 40, 50, 60, 70, 80, 90]`
- **Target:** `80` (at index `7`)
- **Initial pointers:** `left = 0`, `right = 8`

Iteration	`left`	`right`	`mid`	`array[mid]`	Comparison	Action
1	`0`	`8`	`4`	`50`	`50 < 80`	`left = 5`
2	`5`	`8`	`6`	`70`	`70 < 80`	`left = 7`
3	`7`	`8`	`7`	`80`	`80 == 80`	Return index `7`

The target is found in 3 iterations instead of the 8 comparisons a linear search would need.

Comparison Summary

Algorithm	Best Case	Worst Case	Space	Needs Sorted Input
Linear Search	$O(1)$	$O(n)$	$O(1)$	No
Binary Search	$O(1)$	$O(\log n)$	$O(1)$	Yes
Bubble Sort	$O(n)$ with flag	$O(n^2)$	$O(1)$	No

Related Concepts

- Big O Notation: The framework used for every complexity figure above.
- Logarithmic Time: Why halving the search space gives $O(\log n)$.
- Divide and Conquer: The strategy behind binary search and Merge Sort.
- Array: The data structure all of these algorithms operate on.
- Linked List: A structure where binary search is impractical, since it lacks index-based access to the middle element.

Sorting Algortihms

Overview

This note covers Sorting Algorithms. **Selection Sort** repeatedly picks the smallest remaining element and places it at the front of the unsorted section. **Insertion Sort** builds a sorted section one element at a time, shifting larger elements right to make room. Both are compared against Bubble Sort using Big O Notation.

Sorting Algorithms

Sorting means arranging data in a specified order, such as ascending or descending, according to given criteria.

Four main types:

1. **Bubble Sort**
2. **Selection Sort**
3. **Merge Sort**
4. **Insertion Sort**

Bubble Sort

Bubble Sort is one of the simplest sorting methods. It repeatedly compares each element with its **immediate next element** and swaps them if they are in the wrong order. Larger values gradually "bubble up" to the end of the array.

Dry Run

Sorting the array `[4, 19, 1, 3]`:

Pass	Comparison	Action	Array After Step
1	`4` and `19`	Already in order	`[4, 19, 1, 3]`
1	`19` and `1`	`19 > 1`, swap	`[4, 1, 19, 3]`
1	`19` and `3`	`19 > 3`, swap	`[4, 1, 3, 19]`
2	`4` and `1`	`4 > 1`, swap	`[1, 4, 3, 19]`
2	`4` and `3`	`4 > 3`, swap	`[1, 3, 4, 19]`
3	`1` and `3`	Already in order	`[1, 3, 4, 19]`

After pass 1, `19` is locked at the end. After pass 2, `4` is locked next to it. The final result is `[1, 3, 4, 19]`.

Case	Time Complexity	Explanation
Worst case	$O(n^2)$	Nested loops compare nearly every pair, for example on a reversed array
Best case	$O(n)$	Already-sorted input, with an early-exit flag stopping after one pass
Space	$O(1)$	Swaps happen in place using a single `temp` variable

```java
public void bubbleSort(int[] array) {
    // Outer loop tracks how many elements have bubbled to the end
    for (int i = 0; i < array.length - 1; i++) {
        boolean swapped = false; // Tracks whether this pass changed
anything

        // Inner loop compares adjacent elements in the unsorted part
        for (int j = 0; j < array.length - i - 1; j++) {

            // If current element is greater than the NEXT element, swap
them
            if (array[j] > array[j + 1]) {
                int temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
                swapped = true;
            }
        }

        // No swaps means the array is already sorted, so stop early
        if (!swapped) {
            break;
        }
    }
}
```

Why the Flag Matters

Without the `swapped` flag, the loops always run to completion, so even a sorted array costs $O(n^2)$. The flag lets the algorithm detect a pass with zero swaps and exit, which is what produces the $O(n)$ best case.

Selection Sort

Selection Sort divides the array into two parts: a *sorted* side on the left and an *unsorted* side on the right. It repeatedly scans the unsorted part to find the minimum, then swaps it into the first position of the unsorted section.

How It Works

1. Start at the first index (`i = 0`) and assume it holds the smallest value (`min = i`).
2. Use a second loop (`j = i + 1`) to scan the rest of the array to the right.
3. Whenever `array[j] < array[min]`, update `min = j`.
4. After the scan, swap `array[min]` with `array[i]`.

```
public void selectionSort(int[] array) {
    // Outer loop moves the boundary of the unsorted subarray
    for (int i = 0; i < array.length - 1; i++) {
        int min = i; // Assume the current index holds the smallest value

        // Inner loop searches the remaining unsorted elements
        for (int j = i + 1; j < array.length; j++) {
            if (array[j] < array[min]) {
                min = j; // Found a new minimum, save its index
            }
        }

        // Swap the smallest found element into position i
        int temp = array[min];
        array[min] = array[i];
        array[i] = temp;
    }
}
```

Complexity

Case	Time Complexity	Explanation
Best case	$O(n^2)$	Even on a sorted array, the inner loop still scans every remaining element to confirm nothing is smaller
Worst case	$O(n^2)$	Two nested loops scan the array
Space	$O(1)$	Sorts in place using a single `temp` variable

Dry Run

Sorting `[29, 10, 14, 37, 13]`:

Pass	`i`	Scan Result	Swap	Array After Pass
1	`0`	Minimum is `10` at index `1`	Index `0` with index `1`	`[10, 29, 14, 37, 13]`
2	`1`	Minimum is `13` at index `4`	Index `1` with index `4`	`[10, 13, 14, 37, 29]`
3	`2`	Minimum is `14` at index `2`	Index `2` with itself, no change	`[10, 13, 14, 37, 29]`
4	`3`	Minimum is `29` at index `4`	Index `3` with index `4`	`[10, 13, 14, 29, 37]`

The final sorted array is **`[10, 13, 14, 29, 37]`**.

> **Pass 1 Detail**
>
> Starting with `min = 0` (value `29`), the scan finds `10 < 29` at `j = 1`, so `min = 1`. The remaining checks (`14`, `37`, `13`) are none smaller than `10`, so `min` stays at `1`.

Insertion Sort

Insertion Sort is efficient for small or nearly sorted data. It works like sorting playing cards in your hand: pick up one unsorted card (the `key`), shift larger sorted cards to the right, and drop the `key` into the gap.

How It Works

1. Pick the current element as the `key` (`key = array[i]`).
2. Set a pointer `j` to the element immediately to its left (`j = i - 1`).
3. While `j >= 0` and `array[j] > key`, move left (`j--`).
4. Shift each larger value one slot right (`array[j + 1] = array[j]`).
5. When a smaller value or the array start is reached, place the key (`array[j + 1] = key`).

```java
public void insertionSort(int[] array) {
    // Start from the second element; index 0 is a sorted section of size one
    for (int i = 1; i < array.length; i++) {
        int key = array[i]; // Element to insert into the sorted left side
        int j = i - 1;      // Last index of the sorted left side

        // Shift elements greater than key one position to the right
        while (j >= 0 && array[j] > key) {
```

```
            array[j + 1] = array[j];
            j = j - 1;
        }

        // Insert the key into its correct slot
        array[j + 1] = key;
    }
}
```

Loop Bounds

The outer loop must run from `i = 1` to `i < array.length`. Starting at `1` is correct because a single element at index `0` is already sorted. Running to `array.length` ensures the last element is also inserted. A bound of `array.length - 1` would skip it.

Complexity

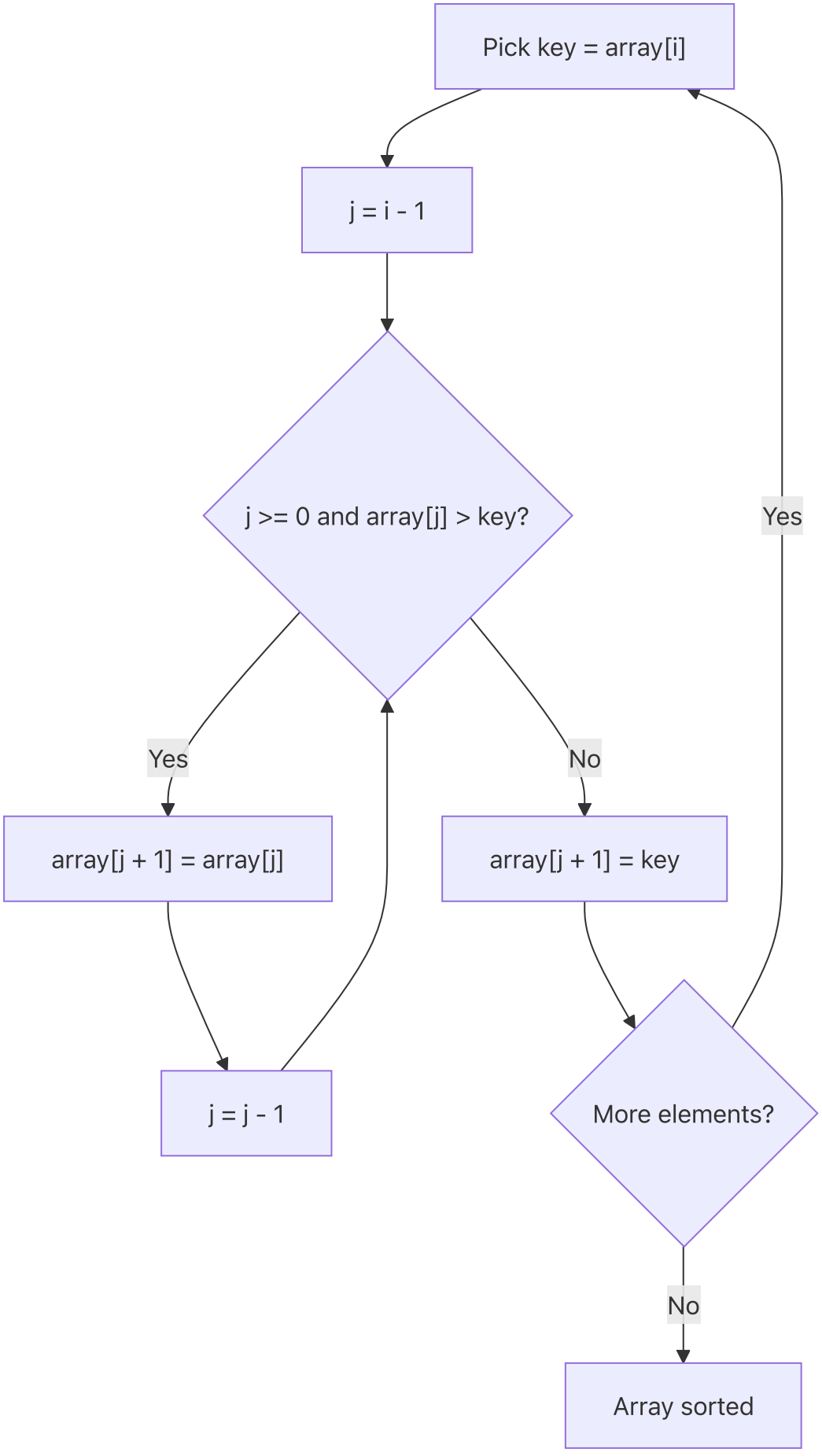
Case	Time Complexity	Explanation
Best case	$O(n)$	On a sorted array, `array[j] > key` is always false, so the `while` loop never runs
Worst case	$O(n^2)$	On a reverse-sorted array, every key shifts all the way back to index `0`
Space	$O(1)$	Sorts in place using only `key` and `j`

Dry Run

Sorting `[8, 3, 5, 2]`, with `[8]` treated as sorted at the start:

Pass	`i`	`key`	Shifts	Insert At	Array After Pass
1	`1`	`3`	`8` moves right	Index `0`	`[3, 8, 5, 2]`
2	`2`	`5`	`8` moves right	Index `1`	`[3, 5, 8, 2]`
3	`3`	`2`	`8`, `5`, `3` move right	Index `0`	`[2, 3, 5, 8]`

The final sorted array is **`[2, 3, 5, 8]`**.



Comparison Summary

Algorithm	Best Case	Worst Case	Space	Key Behavior
Bubble Sort	$O(n)$ with flag	$O(n^2)$	$O(1)$	Swaps adjacent pairs repeatedly
Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Always scans the full unsorted section, so no best-case speedup
Insertion Sort	$O(n)$	$O(n^2)$	$O(1)$	Fast on nearly sorted data

---

Related Concepts

- Sorting Algorithms: The parent topic covering all comparison and ordering strategies.
- Bubble Sort: Another quadratic, in-place sort that shares the same worst case.
- Merge Sort: A faster Divide and Conquer alternative, listed in the sorting overview.
- Big O Notation: The framework behind every complexity figure above.
- Array: The data structure both algorithms operate on in place.